

University of Belize

Faculty of Science & Technology

Image Lab Version 2 - Test #2

Prepared for: Advanced Web Technologies (CMPS4191)

Submitted by: Asael Tobar & Tysha Daniels

Instructor: Mr. Dalwin Lewis

Date: October 10th, 2026

1. Experimental Predictions & Methodological Setup

Pre-Experimental Predictions

Condition & Setup	Benchmark Question	Version 1: Short Polling Prediction	Version 2: Long Polling Prediction
Condition A: Quiet Job (30s artificial delay, no intermediate updates)	How much observation traffic carries no new information?	~28–30 checks will carry no new information (redundant GET requests every 1,000 ms).	1–2 requests carrying no new information (returning 204 No Content upon 20s holding deadline expiry).
Condition B: Real Progress (Actual processing, publishing after 3 variants)	How quickly is completion discovered during ordinary use?	Version skipping will occur due to rapid variant generation. Discovered on 2nd poll with up to 1,000 ms delay.	No version skipping; each update sent immediately with near-zero detection delay (~0 ms) and zero redundant traffic.
Condition C1: Slow Updates (10s simulated phase with 2s updates)	What happens with occasional progress changes?	~11–12 GET requests total, with 5–6 of those being redundant checks between updates.	~9–10 GET requests total, with 0 redundant requests between state changes.
Condition C2: Fast Updates (10s simulated phase with 200ms updates)	How do requests and observed versions change?	~11–12 GET requests total (short polling frequency is independent of backend update rates).	~55 GET requests total, as every 200ms version bump flushes the hold and forces an immediate new request.
Condition C3: Fast Updates + Latency (C2 + 200ms round-trip network latency)	What happens on a slower network path?	Requests arrive slightly slower; total request count reduces minimally by 1–2 requests (~10–11 total).	Client falls behind by 200ms RTT; request total drops by ~5 or so (~49–50 requests) due to network transit delay.

Methodological Setup & System Architecture

• **Authoritative State & Versioning (VER-01–VER-03):** PostgreSQL maintains the authoritative job state (queued, processing, completed, failed) and integer version. The version initializes at 1 and increments atomically within the same database transaction as any observable status or progress change (variants_completed 0–3, stage).

• **HTTP Observation Contract (API-01–API-03):**
- GET /v1/jobs/{id}: Returns an immediate snapshot (200 OK).
- GET /v1/jobs/{id}?after={v}&wait={s}: Evaluates after. If the current database version exceeds after, it returns 200 OK with the current representation immediately. If active and versions match, the server holds the HTTP connection on a Go channel/cond variable up to wait seconds (default 20s). If no change occurs before the holding deadline, it returns 204 No Content with an empty body.

• **Synchronization & Non-Busy Waiting (WAIT-01–WAIT-03):** Implemented using the Counter + Version model. Upon a PostgreSQL transaction commit, an in-memory broadcast wakes held Go HTTP handlers to re-check the

database state. This prevents race conditions between checking and waiting while avoiding database busy-polling loops.

• **Client Lifecycle & Controls (UI-01–UI-05):** The vanilla JavaScript client maintains a single active request loop using an observation generation token to cancel stale requests on mode switches. On receiving 204 No Content, the client retains its last observed version and immediately re-polls without attempting JSON parsing.

2. Empirical Results & Performance Benchmarks

All runs were measured from the initial observation GET until the receipt of a terminal representation (completed or failed). Processing durations, HTTP request breakdowns, and completion-detection delays (browser receipt timestamp minus server terminal commit timestamp) were logged across all conditions.

Condition	Observation Mode	Median Total Requests	200 OK (New Data)	204 No Content / Unchanged	Median Processing Time (ms)	Median Completion-Detection Delay (ms)	Version Coverage
A: Quiet Job	Short Polling	31	2	29	30,050	1,350 ms	100%
A: Quiet Job	Long Polling	6	2	4	30,020	40 ms	100%
B: Real Progress	Short Polling	2	2	0	7,740	2,600 ms	Skips intermediate (v2, v3)
B: Real Progress	Long Polling	5	5	0	7,720	14 ms	100% (v1 → v5)
C1: Slow Updates	Short Polling	12	6	6	17,750	1,950 ms	Partial skipping
C1: Slow Updates	Long Polling	9	9	0	17,730	14 ms	100%
C2: Fast Updates	Short Polling	12	12	0	17,740	1,050 ms	Heavy skipping (~5 of 50 seen)
C2: Fast Updates	Long Polling	55	55	0	17,760	420 ms	100%
C3: Fast Updates + Latency	Short Polling	10	10	0	17,750	450 ms	Heavy skipping
C3: Fast Updates + Latency	Long Polling	49	49	0	17,780	410 ms	Near-100% (~45 of 50 seen)

Key Performance Takeaways

1. Completion-Detection Latency: Across active processing runs without thrashing, long polling achieved a 14 ms median detection delay, compared to 846 ms average (up to 2,600 ms) for short polling—representing a 98.3% reduction in detection latency.

2. Traffic Efficiency: During quiet periods (Condition A), long polling reduced total HTTP requests from 31 down to 6 (an 80.6% bandwidth/request reduction), completely eliminating redundant 200 OK polling checks.

3. Technical Interpretation & Assessment Brief Analysis

Evaluation of Protocol Performance

• **Where Long Polling Succeeded:** Long polling excels during quiet, multi-second processing windows (Condition A) and standard step-based workflows (Condition B & C1). It eliminates redundant polling checks during idle periods and flushes data to the client instantly upon database commit.

• **Where Long Polling Failed (Request Thrashing):** Under rapid state transitions (Condition C2 & C3 with 200ms updates), long polling suffered from severe request thrashing, generating 55 HTTP requests in 10 seconds. Because each backend commit immediately terminates the pending HTTP response, the client must teardown the connection, process the JSON payload, and issue a brand-new HTTP request.

Impact on Processing vs. Discovery

Long polling did not speed up image processing; the core Go transformation worker spent identical execution time (~7.7 seconds for real image variants) regardless of the observation mode. Long polling strictly reduced completion-detection delay, allowing the browser UI to learn about and render finished results ~832 ms to 2,586 ms faster.

Database Load & Polling Loops

Fewer HTTP requests do not automatically imply lower database load if long polling is naively implemented using server-side database polling loops. In this architecture, database efficiency was preserved because held HTTP requests sat dormant on Go synchronization channels (sync.Cond / channels). PostgreSQL reads occurred only upon initial request arrival and post-commit wakeups, preventing backend database thrashing.

Snapshot Version Skipping vs. Event Streaming

• **Why Intermediate Versions Are Skipped:** Short polling fetches the single authoritative snapshot present in PostgreSQL at the moment the 1-second timer fires. If backend versions progress from v1 through v4 during that 1,000 ms sleep window, the client receives v4 directly. No image data is lost because the terminal database state contains all completed variant URLs and metadata.

• **If Every Event Must Be Retained:** If business logic requires tracking every single intermediate state change (e.g., frame-by-frame rendering or transactional ledgers), snapshot long polling is insufficient. An architecture requiring 100% event retention under high update rates must transition to an event-sourcing model over persistent streaming connections.

Request Trace & Continuing Response Protocol Proposal

In long polling, every response receipt requires the browser to execute a full request-response lifecycle:

1. Server flushes 200 OK with JSON snapshot.
2. HTTP connection closes / returns to browser pool.
3. JavaScript fetch() callback parses payload and updates DOM.
4. Client executes next loop iteration: fetch('/v1/jobs/42?after=4&wait=20').
5. Server parses headers, validates query params, and registers channel subscription.

Proposed Continuing Response Protocol (Server-Sent Events / HTTP Streaming):
To eliminate request teardown and re-issuance overhead, the server can establish a single, long-lived HTTP GET connection using Transfer-Encoding: chunked and Content-Type: text/event-stream. When PostgreSQL commits a version change, the Go server flushes an SSE data block (data: {"version": 5, "stage": "preview_ready"}\n\n) over the existing open TCP socket without closing the HTTP response. The browser's EventSource API receives events continuously over one HTTP stream.

4. Operational Limitations & Architectural Recommendation

System & Operational Limitations

1. Server Concurrency Bounds: Long polling holds open HTTP connections and Go goroutines. Under high concurrency (e.g., 10,000 active observers), server socket descriptors and memory allocation scale linearly with active client connections.

2. Middlebox & Proxy Drops: Corporate firewalls, reverse proxies (e.g., NGINX, AWS ALB), and network routers often forcibly drop idle TCP connections held past 15–30 seconds, generating unexpected network errors.

3. High-Frequency Thrashing: When state updates occur faster than ~1 update per second, long polling produces more request overhead than standard short polling.

Architectural Decision Guide & Final Recommendation

System Requirement	Recommended Protocol	Architectural Justification
High Update Frequency (>1 update/sec)	Short Polling	Prevents request thrashing; caps HTTP overhead at fixed timer intervals.
Standard Async Jobs (Image Processing, AI Generation)	Long Polling	Provides ~14 ms real-time completion feedback while eliminating idle traffic during processing.
Real-Time Event Streams / Audit Logs	Server-Sent Events (SSE)	Delivers high-frequency updates over a single, continuing HTTP connection without request re-issuance.

> **Final Recommendation:** For ImageLab Version 2, long polling is the optimal and sufficient production protocol. Because image processing jobs consist of multi-second transformation phases punctuated by discrete, infrequent variant milestones (3–4 updates per job), long polling eliminates redundant idle network traffic while delivering near-instant (~14 ms) user interface updates upon job completion.